

Alien 内核的 x86_64 移植与 vDSO 引入

摘 要

传统内核因为各个模块高度绑定，一处故障很容易扩散到整个系统，导致可靠性降低和维护难度增加。Alien操作系统通过将内核功能拆分为相互隔离的“故障域”，并使用Rust语言的安全机制，在操作系统层面提供更高的容错并缩小漏洞影响范围。x86_64架构在服务器和桌面领域使用广泛、生态成熟，且用户态中断机制已在真实硬件上落地。移植能更好地验证隔离设计的可用性，同时引入新的软件和硬件特性。x86_64在引导、中断控制、Linux ABI与设备发现等机制上与RISC-V存在大量差异，移植到x86_64需要修改整体框架以支持多架构扩展，并添加大量x86_64细节。

为此，本文先分析了Alien原始设计中的域模型，随后设计并实现了x86_64的启动流程、trap/syscall入口、内存管理及设备探测与驱动加载框架。在此过程中，本文提炼出了Trap分发机制等公共抽象，让架构差异仅存在于架构实现内部，使得主逻辑可以跨平台。在此基础上，本文进一步设计并集成了vDSO机制：在保证系统调用路径仍然保留的前提下，通过将一块只读的代码与数据区域映射至用户地址空间，使用户程序获取当前时间可以绕过系统调用。

本文完成了AsyncAlien在x86_64上的包含启动、多核同步、设备发现、域加载及进入用户态的全流程，验证了功能正确性。性能测试结果显示，引入vDSO后，clock_gettime等时间查询操作的开销降低在90%以上。本工作不仅完成了Alien架构移植与功能扩展，也为构建多架构的内核提供了设计参考与实现经验。

关键词：操作系统；多架构支持；内存安全；vDSO；系统调用

Porting the Alien Kernel to x86_64 and Introducing vDSO

Abstract

Traditional kernels suffer from high module coupling, where a single failure can easily propagate throughout the system, reducing reliability and increasing maintenance difficulty. The Alien operating system enhances fault tolerance at the OS level by partitioning kernel functions into isolated "fault domains" and leveraging Rust's safety mechanisms to limit the scope of vulnerabilities. The x86_64 architecture is widely used in servers and desktops with a mature ecosystem, and its user-space interrupt mechanism has been implemented on real hardware. Porting allows for better validation of isolation design feasibility while introducing new software and hardware features. However, x86_64 differs significantly from RISC-V in mechanisms like booting, interrupt control, Linux ABI, and device discovery. Porting to x86_64 requires modifying the overall framework to support multi-architecture extensions and incorporating extensive x86_64-specific details.

To this end, this paper first analyzes the domain model in the original design of Alien, then designs and implements the x86_64 boot process, trap/syscall entry, memory management, and device detection and driver loading framework. During this process, it extracts common abstractions such as the Trap dispatch mechanism, ensuring architectural differences exist only within the implementation of the architecture, allowing the main logic to be cross-platform. Building on this, the paper further designs and integrates the vDSO mechanism: while maintaining the system call path, it maps a read-only code and data region to the user address space, enabling user programs to bypass system calls when retrieving the current time.

This paper completes the full process of AsyncAlien on x86_64, including booting, multi-core synchronization, device discovery, domain loading, and entry into user mode, verifying the correctness of the functions. Performance test results show that with the introduction of vDSO, the overhead of time-related operations such as `clock_gettime` is

reduced by above 90%. This work not only achieves the porting and functional expansion of the Alien architecture but also provides design references and implementation experience for building a multi-architecture kernel.

Key Words: Operating System; Multi-architecture Support; Memory Safety; vDSO; system call

目 录

摘 要	I
Abstract	II
第 1 章 引言	1
1.1 研究背景与意义	1
1.2 研究目标与问题陈述	1
1.3 本文的组织结构	2
第 2 章 背景与相关工作	3
2.1 原始 Alien 系统综述	3
2.2 微内核与隔离设计对比	3
2.3 Rust 语言的内存安全与共享堆实践	4
2.4 vDSO 与高频路径优化	5
第 3 章 系统架构与设计	8
3.1 引入 x86_64 并屏蔽多架构差异	8
3.1.1 引导与早期初始化	8
3.1.2 内存管理	8
3.1.3 中断控制	9
3.1.4 设备探测与域加载	9
3.2 vDSO 设计	10
3.2.1 vdso_crate_template 设计	10
3.2.2 vDSO 与 syscall 设计	11
第 4 章 x86_64 移植与实现细节	12
4.1 引导与早期初始化	12
4.2 x86_64 与 RISC-V 的页表结构与寄存器差异	14
4.3 trap/syscall 初始化	16
4.5 ACPI 与 PCI 的探测方式	20
第 5 章 vDSO 实现与用户态集成	22
5.1 vdso_crate_template 构建与产物	22
5.2 内核侧 MemIf 实现与 vDSO 装载	24

5.3 内核侧刷新策略	25
5.4 用户运行时 vDSO 加载	25
第 6 章 验证与评估	28
6.1 测试设计	28
6.2 功能验证与运行证据	31
6.3 性能度量与分析	32
结 论	33
参考文献	34

第1章 引言

1.1 研究背景与意义

随着计算机系统功能日益复杂，Linux内核代码规模也在不断增长。传统宏内核架构将操作系统的大部分功能集中在单一的执行环境中，任何一个模块出问题，都可能影响整个系统的安全性和可靠性。从多个真实漏洞来看，传统内核的复杂性不只是代码量大、逻辑耦合紧，更导致漏洞修复代价高、验证困难、攻击面宽且难以收敛。

为应对上述风险，Linux内核社区提出了严格的内核内存权限控制、KASLR、seccomp、模块签名等防御手段。这些机制有效提高了攻击门槛，但其本质仍是对复杂系统的被动补救，难以从根本上降低内核复杂性带来的风险。隔离式内核架构由此成为一种可行的解决方案：通过把系统功能划分为相对独立的故障隔离域，各域通过明确接口交互，避免直接共享内核内存，从而将故障影响局限在域内并减少受影响的内核路径。

在这种架构下，系统具备如下优势：

1. 故障局限化：单个域的漏洞或故障不会导致整个系统崩溃
2. 攻击面缩减：每个域只暴露必要的接口，减少高风险路径
3. 独立演进：功能模块可以独立升级或替换，降低系统整体维护成本
4. 隔离边界明确：安全属性更易于验证与审计

Alien是基于域隔离理念的内核实现，已在RISC-V平台验证了隔离域、共享堆通信与域加载等机制。x86_64平台在启动、trap处理、内存和设备管理等方面与RISC-V存在显著差异，需要设计平台适配层。系统调用、时间查询等高频路径对性能尤为敏感，传统Linux通过vDSO在用户态提供快速路径以降低开销。任务调度框架作为操作系统的核心组件之一，对于任务整体性能有着至关重要的作用。为了引入基于vDSO的调度框架vsched2，需要先引入vDSO以及快速syscall路径作为前置工作。

1.2 研究目标与问题陈述

本文的研究目标是：在不动现有RISC-V路径的前提下，打通x86_64的运行链路，使系统能够完成启动、早期初始化、设备发现、域加载、多核启动并最终进入用户态；

同时为vDSO、协作调度等高频机制预留好扩展接口，让这个最小可运行版本后续有优化余地。

围绕这个目标，本文需要回答以下2个问题：

- 如何在启动流程、内存布局、中断处理、设备探测机制等方面建立统一的抽象，使x86_64与RISC-V之间实现尽可能多的代码复用，同时保持语义和功能 correctness？
- 如何将vDSO引入Alien，同时不影响现有的域隔离、系统调用等机制，实现一个功能正确的时间获取调用？

本文的研究范围就限定在上面这些问题在x86_64平台上的实现和验证。最后会从功能正确性和性能开销这2个维度，说明迁移后的系统不仅能启动，而且具备一定的实际可用性。

1.3 本文的组织结构

本文后续章节按背景、实现、验证的顺序展开。

第二章介绍Alien的原始设计以及相关研究进展，包含域隔离、共享堆通信、vDSO和协作调度等核心概念，为后面的设计提供理论基础。

第三章给出系统整体架构与设计要点，说明平台抽象、虚拟地址空间和高频路径的统一设计思路，以及如何在设计中为vDSO和协作调度预留扩展接口。

第四章重点讲x86_64的移植和实现细节，依次说明引导与早期初始化、物理与虚拟内存管理、多核启动同步、trap与syscall的适配、任务切换、设备发现以及驱动域加载的工程实现和关键取舍。

第五章描述vDSO的构建、vVAR布局与无锁一致性策略、用户态映射与auxv注入，以及快路径和回退机制的协同实现。

第六章给出实验验证与评估方法，包括功能正确性测试、隔离性与恢复性验证，以及针对syscall和vDSO的性能基准测试与分析。

第2章 背景与相关工作

2.1 原始 Alien 系统综述

Alien[1]的核心思路不是简单的模块化，而是把内核能力按职责和功能拆成多个故障隔离域，再让它们通过显式接口协作。传统宏内核是“各模块共享同一个高权限执行空间”，Alien的设计则直接在框架上把访问边界、调用边界和故障边界划分清楚，这样系统在局部出问题，其它部分不容易受影响，大概率仅需要修复局部即可。本文的迁移工作不能破坏这个设计，并且应优先复用那些已经验证过的域运行时机制。

从运行角度来看，Alien 的内核功能以一组可动态管理的隔离域形式存在，这些域作为内核模块支持运行时加载、卸载与热更新。每个域通过显式的域接口对外提供服务，域内部依赖安全抽象实现其功能；域管理运行时负责域的装载、回收与替换等治理动作，从而在运行时形成“加载—通信—治理”的闭环。该闭环把隔离、故障隔离与演进能力作为设计核心：通过语言级的类型与内存安全约束和清晰的接口边界，既允许域独立演进与热替换，又避免跨域协作退化为难以观测和验证的隐式耦合。

在域间通信上，Alien采用控制面和数据面分离的方式。控制路径使用受限接口和轻量调用，数据路径借助共享堆来传递对象。这种设计兼顾了性能和边界可验证性：一方面降低了高频数据交换的复制成本，另一方面通过受控类型和生命周期约束，减少了直接共享内存带来的越界访问和对象悬挂风险。Alien不依赖人工约定来维持正确性，而是尽量把约束落到接口和类型边界上。

不过，原始Alien的实现主要面向RISC-V平台，它的启动链路、异常入口、设备发现和中断模型都带有明显的平台特征。这在原型阶段有利于快速验证域机制的可行性，但同时也导致部分实现与平台细节耦合较深。因此，本文所做的x86_64迁移并不是另起炉灶，而是在保留域机制主干的前提下，把平台相关差异从上层运行时中收敛出去，逐步建立起一套可复用的跨架构实现骨架。

2.2 微内核与隔离设计对比

微内核体系[14]（如seL4[13]）通过最小化内核功能来压缩可信计算基，并依赖消息传递实现服务协作，隔离边界天然更强。与此相对，Alien采取的路线更偏向工程折中：不追求一次性重写全部系统服务到用户态，而是在现有内核形态上逐步引入

隔离域和显式接口，采用“渐进式隔离重构”的策略。两者的共同目标是降低故障扩散和攻击面，不同点在于实施路径与成本结构。

从设计粒度看，微内核追求最小内核，通常只保留地址空间、线程和进程间通信等最基础的服务，其他功能以服务进程形式运行在用户态。这种设计天然具有强隔离但引入的上下文切换和消息传递开销也较高。**Alien**则保留了大部分内核功能在内核态执行，但通过域机制加强了模块间的边界与可验证性，兼顾了隔离效果与性能开销。

对本研究来说，微内核提供了隔离设计的参照系与终极目标，而**Alien**路线强调迁移可行性与渐进演化能力，更贴合当前代码基础与实践条件。这一权衡决定了后续x86_64迁移的实施策略：优先复用现有运行时框架，在其上引入平台抽象和隔离边界，而非彻底推翻重来。

2.3 Rust 语言的内存安全与共享堆实践

Rust的所有权、借用和生命周期机制[16]为隔离域系统提供了更坚实的静态约束基础。传统用C实现的隔离系统往往需要开发者手工维护引用计数、对象归属关系和访问权限，一旦出现疏忽，错误就难以追踪和复现。而在**Alien**中，共享堆通信并不意味着“随意共享内存”，而是通过受控数据结构和接口边界来传递对象，尽可能把对象归属、可变性和回收责任直接写进类型语义，由编译器强制执行。这一实践能够显著降低双重释放、悬空引用和越界访问等常见内存错误的发生概率。从编译器视角看，**Rust**的类型系统和生命周期检查能够在编译期就捕获大量内存安全问题，避免将崩溃或未定义行为留到运行时暴露。这一点对隔离域系统尤为关键：域间对象传递不能仅依赖运行时检查，必须尽可能在编译期形成强保证。例如，当某个对象从域A转移到域B时，**Rust**的所有权转移规则在编译阶段就保证了域A不可能再继续访问该对象，由此从根本上消除了域间因对象共享而产生的竞态条件和悬挂指针。当然，**Rust**并不表示底层代码就可以完全摆脱不安全操作。域加载、地址空间映射、硬件直接交互等路径仍然需要少量unsafe代码，这些代码触及底层内存和硬件语义，已经超出了**Rust**编译器安全覆盖的范围。相关实践的意义在于表明：**Rust**能够将绝大多数可能引入错误的代码压缩到少数几个可集中审计的边界内，从而使隔离域系统的工程质量更加可控，而不是宣称“语言本身会自动解决所有问题”。这样的分层设计让后续的代码审计、测试与形式化验证都能够更加聚焦有效。

2.4 vDSO 与高频路径优化

vDSO（virtual dynamic shared object）[12]是Linux内核为降低高频系统调用开销而引入的一项经典优化机制。传统系统调用的性能瓶颈主要体现在用户态与内核态之间的上下文切换开销——每次陷入内核都需要保存和恢复大量寄存器状态，这一固定成本在调用频率极高时将累积为不可忽视的性能损失。vDSO的设计目标正是在保持语义正确的前提下，让某些无需修改内核状态的系统调用完全在用户态执行，从而消除陷入内核的开销。

vDSO的起源可以追溯到2005年前后Linux2.6内核引入的vsyscall机制，后者是Linux最早的加速系统调用的尝试，出现在2.6内核中，通过将少量内核代码以静态页面的形式映射到用户地址空间来实现快速调用。然而vsyscall存在若干固有缺陷，其采用固定静态地址映射，缺乏灵活性，也无法为不同进程提供差异化的映射策略。作为改进方案，vDSO在2007年由Andi Kleen首次在x86-64架构上实现[18]，内核将一小段ELF共享目标文件自动映射到每个用户进程的地址空间，该文件包含了gettimeofday和clock_gettime等高频系统调用的用户态实现。相较于vsyscall的静态内存映射方式，vDSO借助ELF共享目标文件天然的PIC（位置无关代码）特性，以进程为单位动态映射到不同地址空间中，同时为每个进程随机化映射地址以增强安全性。

vDSO在性能上带来的提升是显著的。以经典的gettimeofday调用为例，在s390平台上进行的一亿次调用基准测试中，纯系统调用路径耗时8.6秒，而vDSO路径仅需1.0至1.3秒，性能提升约6至8倍[20]。在arm64平台上，arm64vDSO的引入使clock_gettime-monotonic的耗时降至原先的23%，gettimeofday降至26%，其他时钟相关调用也有3倍以上的改善。在x86_64平台上，CLOCK_MONOTONIC路径的vDSO调用平均延迟约25纳秒且无需陷入内核，而强制走系统调用路径则需约320纳秒，差异超过一个数量级。

随着vDSO在多架构上的广泛部署，其实现分散在各架构目录中也带来了维护负担。2019年，Vincenzo Frascino在Linux5.2开发周期中提交了通用vDSO实现[19]，将大量架构相关代码重构为统一的通用层，使vDSO可以跨越x86、arm64、s390、MIPS等多种架构共享同一套核心逻辑。这一工作的意义不仅在于减少了代码重复，更重要的是确立了vDSO的跨架构设计模式：内核将只读的vVAR数据页映射到用户空间，

vDSO代码通过读取这些数据页中的预更新值（如时钟调整参数）来完成快速计算，从而在不进入内核的情况下返回正确结果。

与此同时，vDSO的安全性始终是学术界和工业界关注的焦点。从安全模型来看，vDSO代码完全运行在用户空间，并不跨越任何安全边界；当需要触及内核特权操作时，vDSO必须通过系统调用回退到内核执行。然而历史上vDSO仍暴露出若干安全问题。早期vDSO映射地址固定，曾被用于绕过ASLR（地址空间布局随机化）保护，攻击者可通过猜测vDSO的固定地址来构造ROP链。此外，vDSO区域还可能被用作潜在的KASLR侧信道信息泄露源，攻击者可通过推测侧信道从vDSO映射地址推导内核基址。vDSO劫持技术甚至被用于容器逃逸，例如DirtyCOW漏洞通过篡改vDSO代码突破容器隔离获取宿主机root权限[22]。针对这些安全威胁，内核社区持续加固vDSO的安全防护：通过地址随机化消除固定地址攻击面；在Linux6.15中引入mseal机制，将vdso、vvar、sigpage等系统映射锁定为只读或仅可执行，确保它们在进程生命周期内不被篡改或解除映射[21]。

从vDSO的技术演进中，可以提炼出几条具有普遍参考价值的设计原则。其一，“快慢路径协同”——高频、无状态、低风险的操作优先通过用户态快速路径完成以降低延迟，当快速路径条件不满足或一致性检查失败时，则无缝回退到系统调用慢路径以保证语义正确性。其二，快速路径与主链路不可割裂——若vDSO快速路径与调度器维护的系统时间源不同步，可能导致应用层观察到时间倒流或跳跃，因此快速路径的正确性不仅取决于自身算法，还要求与整个系统的时间管理、调度和一致性框架紧密协同。其三，安全加固应嵌入机制设计而非事后弥补——地址随机化、内存密封等措施已经证明，将安全约束固化到vDSO的基础机制中远比打补丁更可靠。

对Alien而言，vDSO不能被视为独立优化补丁，而应与域隔离、系统调用回退和调度语义协同设计。若快速路径与主链路割裂，可能带来语义偏差甚至边界绕过风险。例如，若vDSO中的时间查询与调度器维护的系统时间不一致，可能导致应用层观察到时间倒流；若vDSO快速路径绕过了域隔离边界，则也就破坏了隔离承诺。因此，vDSO的正确性不仅要求快速路径算法本身正确，还要求与整个系统的隔离、调度和一致性框架协同。

本研究借鉴vDSO思想的重点并非“照搬实现”，而是建立“快慢路径协同”的设计原则。这一原则意味着：高频、低风险、状态不变的服务优先走用户态快速路径以降低延迟；当快速路径条件不满足（如需要修改状态或缺乏必要信息）或一致性检查失

败时，必须无缝回退到系统调用慢路径并由内核完成。这样的设计既保证了性能收益，也保证了语义正确性和域隔离不被绕过。该原则为后续x86_64实现章节提供了性能与正确性并重的设计基线。

第3章 系统架构与设计

本文在这一章给出总体架构和关键设计要点，目标是在尽可能屏蔽架构差异的情况下引入x86_64平台相关实现，完成x86_64运行链路，然后在此基础上实现vDSO的引入和测试。

3.1 引入 x86_64 并屏蔽多架构差异

设计目标是把架构相关的差异限制在可控的几处，从而使上层业务逻辑在统一的抽象之上工作。为此把移植工作拆分为四部分：引导与早期初始化、内存管理、中断控制、设备探测与域绑定。每一部分收敛平台差异，向上层提供统一接口，内部按架构实现。

3.1.1 引导与早期初始化

引导层的策略是让不同平台通过不同的启动汇编进入统一的Rust编写的main函数，并传递引导信息指针。具体做法是约定一个统一的boot_info参数，传递物理内存布局、initrd基址、可用内存信息等数据，留给后续部分使用。这个信息以指针的形式存在，在不同平台上可能有不同的实际含义。在x86_64上，Multiboot引导完成后处于32位保护模式，需要设置页表，设置CR0、CR4、EFER控制寄存器进入长模式，然后再进入main；而RISC-V侧只需要在SBI初始化完成后压入boot信息指针后进入main即可。随后双架构初始化percpu数据寄存器（x86_64上是gs，RISC-V上是gp）并进入早期初始化。x86_64侧在早期初始化中设置APIC打开本地中断、启用浮点/SSE功能，建立早期时间模块。而RISC-V得益于其简洁的设计，不需要进行相关工作。早期初始化的设计将更多平台差异隐藏到具体实现中。

3.1.2 内存管理

在物理内存管理上，x86_64使用Multiboot信息获取可用物理内存区域，RISC-V解析FDT的/memory节点获取可用物理内存信息，在去除低地址区域和内核ELF占据的物理页后，物理页帧管理器将剩余可用物理内存按页划分，构建空闲页帧池，并提供统一的分配与回收接口。物理页帧回收工作可以借助Rust语言提供的RAII特性进行自动回收。内核链接脚本中预留了内核堆空间，这部分物理内存交给内核堆分配器进行分配。由于内核堆空间在内核ELF区域内，故不会与物理页帧分配器冲突。

在虚拟内存管理上，核心是定义一个跨架构的VSpace抽象，包含如下操作：获取页表根地址、创建/销毁地址空间、映射与取消物理页映射、查询虚拟地址到物理地址的映射。每种架构提供对应的页表项和页表寄存器实现：在riscv上，适配器实现Sv39语义并通过satp切换页表根；在x86_64上，适配器实现四级页表语义并通过CR3切换页表根。上层使用了VSpace抽象操作而不是直接操作寄存器或页表条目，从而屏蔽页表层级和寄存器的差异。域加载、vDSO映射以及用户地址空间的初始化都通过该抽象完成。

3.1.3 中断控制

中断控制采用两层设计：汇编入口负责保存和恢复中断上下文、切换栈和地址空间，然后把控制权交给用Rust实现的中断分发函数。对于RISC-V，故障与系统调用共享中断路径，仅需要区分内核中断与用户中断路径。x86_64引入了syscall指令和syscall MSR，需要额外区分故障和系统调用路径。故障处理时，部分寄存器的保存和恢复由CPU自动完成，剩余逻辑与RISC-V类似。系统调用则完全由软件保存和恢复所有寄存器，本文的做法是将percpu区域映射到用户态高地址，先暂存用户栈指针到内存中再恢复，以此完成换栈操作。中断分发函数负责解码异常原因、调用相应的处理逻辑并将结果写入用于恢复的中断上下文。这部分受架构影响较大，分别实现更好。

3.1.4 设备探测与域加载

设备探测职责在于把平台发现设备的细节隐藏到平台实现中，对外只暴露出探测得到的设备信息。实现上，RISC-V使用FDT进行设备解析，x86_64则使用ACPI/PCI机制。不同平台存在不同平台的专有设备，例如PLIC、Goldfish等，即使是相同设备也存在不同的I/O方式，例如MMIO和PIO。在本文的设计中，对于平台专有设备的驱动域，假定在指定架构下实现，其它架构尝试编译会报错；对于平台共用设备的驱动域，其实现中支持多种I/O方式，域加载时需要按架构初始化I/O方式，这样就能隐藏架构差异，复用同一段业务逻辑代码。上述两类域需要判断当前架构后加载。对于内核组件域中架构相关的部分，在实现中统一域接口操作，内部按架构实现。除此之外，还存在大量架构无关的内核组件域和虚拟设备域，这些域可以直接加载。由此便完成了设备探测与域加载的平台无关上层抽象。

总体上，引入x86_64的关键在于把架构专有步骤封装在少数几个边界：引导和早期初始化、VSpace内部实现、汇编中断入口和中断分发函数、设备探测实现与域加载架构专有路径。只要这些边界的约定清晰且稳定，上层业务逻辑便无需了解底层的寄存器、页表或异常细节，从而实现跨架构的代码复用与可维护性。

3.2 vDSO 设计

本小节分为两部分：首先说明vdso_crate_template的设计目标与生成器职责，然后论述vDSO与syscall的设计与回退语义。

3.2.1 vdso_crate_template 设计

vdso_crate_template[6]的核心职责是生成符合vDSO[12]要求的动态链接库，并提供内核和用户库的加载和访问接口。它不直接承担具体的物理内存和用户页表管理，而是把这些职责抽象为接口，由内核去完成。

在vdso_crate_template的设计中，数据分为共享数据和私有数据。从地址空间的角度看，共享数据可以被所有地址空间访问，而每个地址空间只能访问自己的私有数据。

生成器在构建时输出两类主要结果，即vDSO共享对象和静态API库。内核可以通过API库自动处理vDSO代码和数据的加载和重定位。用户库，可以通过API库像普通函数调用一样调用vDSO函数，因为API库记录了函数API及其在vDSO的偏移，只要用户提供了auxv提供的vDSO起始地址，就可以计算出vDSO函数地址。

在实际映射流程中，生成器通过一个内存接口由内核完成地址与物理页的分配与映射。这个接口把vDSO加载过程抽象出分配虚拟地址、分配物理页、页映射、权限更改和虚拟地址翻译这几个内核实现，使生成器逻辑能够同时适配内核初始化与映射到用户地址空间两种场景。

MemIf接口与功能对应如下：

表 3-1 MemIf 接口设计

接口名	功能说明
MemIf::valloc	在指定地址空间中预留一段虚拟地址区域供 vVAR 与 vDSO 使用，不立即分配物理页，返回该区域的基址。
MemIf::ppage_alloc	分配连续的或若干物理页用于存放 vVAR 或 vDSO 数据，返回物理页句柄。
MemIf::ppage_clone	复制物理页句柄以便在不同地址空间中安全地复用相同物理页，或按实现进行引用计数处理。

MemIf::map	将指定的物理页句柄映射到目标地址空间的虚地址，并设置只读、可执行、用户可见等权限标志。
MemIf::change_protect	在映射完成后修改该虚地址区间的权限，用于把 vVAR 先以可写方式初始化后切换为只读，或把代码段设为只读可执行。
MemIf::get_kernel_vaddr	在内核上下文中获取映射到用户地址的内核可访问虚拟地址，便于内核在初始化阶段直接向 vVAR 写入数据或验证映射内容。

内核端在首次加载时完成完整的ELF解析、段拷贝和重定位，在后续为其他地址空间建立映射时可以复用已加载的物理页。为了兼容物理页句柄通过Drop trait自动释放的设计，MemIf还提供了ppage_clone用于延续物理页生命周期。

3.2.2 vDSO 与 syscall 设计

本文为测试vDSO功能，在vDSO中设计了clock_gettime和gettimeofday的快速路径实现。这两个函数不存在私有状态，全部状态均存在于共享区。

在clock_gettime的总体设计中，vDSO内部只负责从共享区中读取当前时间并把结果转换为用户态所需的格式，不参与时间维护。

gettimeofday与clock_gettime共享同一套时间快照，实现逻辑类似，都是只从共享区内读取时间值，转换为对应的时间格式。这也是实现这两个系统调用的原因。

内核侧的刷新逻辑负责维护这份共享时间快照。内核在时钟中断读取当前时间，再把最新值写入共享区，并通过原子变量标记告知用户是否正在更新。用户态读取时先检查更新是否完成，再读取时间并在必要时重试，从而避免读到更新中的脏数据。由此，clock_gettime和gettimeofday在正常情况下都可以直接命中vDSO快路径，而当共享区暂不可用、内核正在更新中或其它异常情况时，统一回退到syscall，保证语义完整和运行稳定。

内核在进程初始化时先建立vDSO和共享时间区域的映射，再通过auxv把vDSO入口暴露给用户态解析，用户态进行缓存。随后内核持续刷新共享时间快照，使用户态始终能够通过同一套vDSO接口获得较低开销的时间查询结果。

第 4 章 x86_64 移植与实现细节

本章描述将系统从RISC-V迁移到x86_64时的实现要点，聚焦启动与早期初始化、percpu与早期初始化、页表与寄存器差异、trap/syscall的切换语义、以及ACPI/PCI的探测方法。

4.1 引导与早期初始化

为了使用Multiboot进行引导，首先按照Multiboot1规范[10]，通过链接脚本控制在二进制文件头部嵌入一个Multiboot header，指定ELF加载范围，以及入口点。x86_64的早期启动需启用分页模式，加载早期页表，完成控制寄存器与长模式相关配置，最后ljmp进入64位代码。Multiboot引导完成后，eax中存储着魔数，ebx中是Multiboot信息指针。根据x86_64 ABI，将魔数作为第1个参数移到edi，Multiboot指针作为第2个参数移到esi后，就可以调用Rust函数，校验魔数后进入Rust编写的主函数。

第一步是初始化percpu寄存器和cpuid。引入percpu crate需要在链接脚本中添加一个percpu段。percpu的初始化在预先分配的内存中构造每核数据区并设置percpu寄存器值，以便在陷阱/系统调用时通过GS快速访问。cpuid在RISC-V侧SBI引导后通过寄存器传递给entry，而x86_64侧可以通过封装底层cpuid汇编操作的raw_cpuid crate获取cpuid，然后写入percpu区。自此之后，所有的cpuid获取都通过percpu区获取。

接下来是平台早期初始化。对于x86_64，每个CPU在进入内核运行之前需要初始化FPU/SSE[4]支持并配置本地APIC功能。

表 4-1 FPU/SSE 初始化代码

```
pub fn init_fpu() {
    unsafe {
        // CR0: 清 EM/TS, 置 MP。
        let mut cr0 = control::Cr0::read();
        cr0.remove(control::Cr0Flags::EMULATE_COPROCESSOR);
        cr0.insert(control::Cr0Flags::MONITOR_COPROCESSOR);
        cr0.remove(control::Cr0Flags::TASK_SWITCHED);
        control::Cr0::write(cr0);

        // CR4: 置 OSFXSR/OSXMMEXCPT。
        let mut cr4 = control::Cr4::read();
        cr4.insert(control::Cr4Flags::OSFXSR);
        cr4.insert(control::Cr4Flags::OSXMMEXCPT_ENABLE);
    }
}
```

```

control::Cr4::write(cr4);

// 保留 x87 初始化。
core::arch::asm!("fninit", options(nomem, nostack,
preserves_flags));
}
}

```

首先在CR0上清除FPU模拟位并设置MP，允许协处理器按x86语义工作；随后通过CR4启用OSFXSR与OSXMMEXCPT，从而支持使用FXSAVE/FXRSTOR与SSE的异常模型。执行fninit将x87状态复位为已知值，保证后续浮点指令在已定义的状态下运行。

表 4-2 主核 APIC 初始化代码

```

/// 初始化主核（BSP）的 Local APIC。
pub fn init_primary_apic() {
    // 禁用 8259A PIC
    let _ = x86_apic::port_io::disable_8259a();

    let is_x2apic_mode = cpu_has_x2apic();
    IS_X2APIC.store(is_x2apic_mode, Ordering::Release);

    let xapic_base = if is_x2apic_mode {
        0 // x2APIC 不需要 MMIO 基地址
    } else {
        (unsafe { x2apic::lapic::xapic_base() as usize }) +
        (PHYS_VIRT_OFFSET as usize)
    };

    let mut context = LocalApicContext::new(xapic_base,
is_x2apic_mode)
        .expect("failed to create LocalApicContext");

    // 启用 APIC
    context.enable().expect("failed to enable APIC");

    *APIC_CONTEXT.lock() = Some(context);
    APIC_CONTEXT_READY.store(true, Ordering::Release);
}

```

init_primary_apic在主核上完成 Local APIC 的早期初始化：先通过x86_apic::port_io::disable_8259a()关闭旧式8259A PIC，避免干扰。随后通过

`cpu_has_x2apic()`检测CPU是否支持x2APIC，并把结果写入全局变量`IS_X2APIC`。接下来根据是否启用x2APIC决定`xapic_base`。x2APIC模式不需要MMIO，而是通过MSR访问，因此将`xapic_base`设置为0。否则从`x2apic::lapic::xapic_base()`获取物理地址并加上`PHYS_VIRT_OFFSET`转换为内核虚拟地址。用该基址和模式创建`LocalApicContext`（在`x2apic crate[3]`基础上对APIC的进一步封装），调用`context.enable()`对硬件做寄存器编程，例如设置`timer vector`、配置`timer`等。最后将上下文写入互斥锁保护的`APIC_CONTEXT`并通过原子变量`APIC_CONTEXT_READY`更新就绪状态。

在`x2apic crate`的设计中，`LocalApic`对象仅记录设置项，只有在调用`enable()/enable_timer()`时才会真正写入寄存器。因此，主核创建`LocalApicContext`对象后，从核可以直接在这个对象上调用`enable()`，对自己核上的APIC进行配置。

4.2 x86_64 与 RISC-V 的页表结构与寄存器差异

x86_64使用四级页表（PML4→PDPT→PD→PT），每级512项，索引宽度为9位。RISC-V的Sv39使用三级页表（PGD→PMD→PT），每级512项，索引宽度为9位。

表 4-3 x86_64 与 RISC-V 页表项和页表抽象

```
#[cfg(target_arch = "x86_64")]
pub type ArchPTE = X64PTE;
#[cfg(not(target_arch = "x86_64"))]
pub type ArchPTE = Rv64PTE;

#[cfg(target_arch = "x86_64")]
pub type ArchPageTable<T> = X64PageTable<T>;
#[cfg(not(target_arch = "x86_64"))]
pub type ArchPageTable<T> = Sv39PageTable<T>;

pub struct VmSpace<T: PagingIf<ArchPTE>> {
    table: ArchPageTable<T>,
    areas: BTreeMap<usize, VmAreaType>,
}

pub trait PagingIf<PTE: GenericPTE>: Sized {
    /// 分配 4K 大小的物理页面
    fn alloc_frame() -> Option<Box<dyn NotLeafPage<PTE>>>;
}
```

对于页表项，可以抽象出来的主要操作有：

1. 创建一个指向指定物理页的页表项
2. 设置页表项的权限
3. 获取页表项指向的物理地址和权限

而对于页表，可以抽象出来的主要操作有：

1. 映射虚拟页A到物理页B
2. 修改虚拟页A映射的物理页B
3. 修改虚拟页A映射权限
4. 删除虚拟页A映射
5. 获取页表根目录的物理地址
6. 查询虚拟地址映射的物理地址

在page-table crate的设计下，不同架构可以抽象出相同的页表操作，并进一步通过VmSpace与PagingIf接口实现基于RAII的物理页面分配和自动释放。在VmSpace对象触发drop时，内部包含的通过PageingIf::alloc分配的物理页对象将会自动drop，返还占用的物理地址。

对于不同架构，使用的页表寄存器也各不相同，x86_64使用CR3寄存器，RISC-V使用satp寄存器。切换页表时写入页表寄存器的值的含义也各不相同。

表 4-4 获取写入页表寄存器的值

```
pub fn kernel_page_table_token() -> usize {
    #[cfg(target_arch = "x86_64")]
    {
        kernel_page_table_root_paddr()
    }
    #[cfg(target_arch = "riscv64")]
    {
        8usize << 60 | (kernel_page_table_root_paddr() >>
FRAME_BITS)
    }
}

/// 激活页表（Sv39）。
#[cfg(target_arch = "riscv64")]
pub fn activate_paging_mode(page_table_token: usize) {
    sfence_vma_all();
    satp::write(page_table_token);
    sfence_vma_all();
}
```

```
// 激活页表
#[cfg(target_arch = "x86_64")]
pub fn activate_paging_mode(page_table_token: usize) {
    unsafe {
        Cr3::write(
            PhysFrame::containing_address(
                PhysAddr::new(page_table_token as u64)
            ),
            Cr3Flags::empty()
        );
    }
}
```

如代码所示，在x86_64上，`kernel_page_table_token`返回值直接是内核页表根的物理地址，上层在切换CR3时使用该物理地址即可。在riscv64上，函数将物理页号与标志（这里代表使用Sv39分页模式）合并成一个token，然后写入satp寄存器。对于上层，可以直接使用`activate_paging_mode(kernel_page_table_token())`激活，而不需要关注实现细节。

4.3 trap/syscall 初始化

在x86_64中，中断初始化需要设置trap和syscall两条路径。由于Alien在RISC-V中的实现就是使用跳板页完成内核和用户地址空间的切换，本文在x86_64下也保留类似的实现。

表 4-5 trap 汇编入口

```
# 为每个中断号定义 handler
.altmacro
.macro DEF_HANDLER, i
.Ltrap_handler_\i:
.if \i == 8 || (\i >= 10 && \i <= 14) || \i == 17
    # CPU 已压入 error_code
    push \i
    jmp .Ltrap_common
.else
    # 统一补齐 error_code, 保持 TrapFrame 布局一致
    push 0
    push \i
    jmp .Ltrap_common
.endif
```

```
.endm

.Ltrap_common:
    push rax
    ... (保存剩余通用寄存器)

    # 统一入口：根据保存的 CS.DPL 分流内核态/用户态路径。
    mov rax, [rsp + TF_CS]
    and rax, 0x3
    cmp rax, 0x3
    je .Lfrom_user

    ... (内核 trap 分发函数调用)

    pop r15
    ... (恢复剩余通用寄存器)
    iretq

.Lfrom_user:
    # 用户态进入内核后切到内核 GS 基址 (percpu)。
    swapgs

    # 从任务 TrapFrame 取内核 CR3/栈。
    # 注意：切 CR3 前先取完所需值，避免切换后地址不可见。
    mov r14, [rsp + TF_K_SP]
    mov r15, [rsp + TF_K_CR3]
    mov cr3, r15
    # 从此处开始进入内核地址空间，rsp 切换到 k_sp，指向内核栈
    # 顶。
    mov rsp, r14

    ... (用户 trap 分发函数调用)

    # 恢复用户寄存器
    ...
    pop rax

    # 恢复用户 GS 基址
    swapgs

    ...
    iretq
```

在x86_64中，中断处理函数通过lidt指令写入中断描述符表（IDT）地址的方式实现。本文为每个中断号设置一个处理函数，在开头压入中断号，随后复用同一段公共代码，在Rust编写中断分发函数中按不同路径处理。对于8，10-14，17这几个硬件异常，CPU会自动压入错误码，其它情况下压入1个0维持trap上下文结构一致。本文在保存通用寄存器后，通过CS寄存器判断中断来源特权级。若来自用户态，则执行swaps交换IA32_GS_BASE和IA32_KERNEL_GS_BASE的值，切换到内核gs，再加载内核cr3和rsp切换到内核地址空间和内核栈，否则跳过这些操作。返回到用户时，由Rust代码向汇编传递用户cr3和trap上下文虚拟地址。切换到用户地址空间后，将rsp指向trap上下文虚拟地址，然后弹出通用寄存器，最后执行swaps切换到用户gs并通过iretq返回。返回到内核时，rsp自然指向trap上下文，只需要恢复通用寄存器后执行iretq即可。

为了在用户地址空间下仍能在syscall中访问percpu数据，内核在所有虚拟地址空间的高地址映射一份percpu镜像并在初始化时把percpu寄存器设为该镜像的基址。在切换地址空间时，不需要修改GS的值就能正确访问percpu数据。

表 4-6 percpu 镜像写入与 LSTAR/SFMask 设置

```
// 在 syscall 初始化时把 GS 指向 percpu 镜像高地址
let gs_value = percpu::read_percpu_reg();
let gs_mirror = PERCPU_MIRROR_BASE - _percpu_start as *const () as usize
+ gs_value;
unsafe { percpu::write_percpu_reg(gs_mirror); }

// 设置 syscall 相关 MSR，将快速入口指向 trampoline 内的 syscall_entry
let lstar_offset = syscall_entry as *const () as usize - strampoline as
*const () as usize;
let lstar = VirtAddr::new((TRAMPOLINE + lstar_offset) as u64);
LStar::write(lstar);
// 屏蔽 TF/IF/DF/IOPL/NT/AC，避免带入用户态标志位。
let sfmask = RFlags::TRAP_FLAG
    | RFlags::INTERRUPT_FLAG
    | RFlags::DIRECTION_FLAG
    | RFlags::IOPL_LOW
    | RFlags::IOPL_HIGH
    | RFlags::NESTED_TASK
    | RFlags::ALIGNMENT_CHECK;
SFMask::write(sfmask);
unsafe { Efer::write(Efer::read() |
```

```
EferFlags::SYSTEM_CALL_EXTENSIONS); }
```

上述代码片段在内核虚拟地址空间为percpu数据建立一个稳定且可由汇编访问的高地址镜像，并将该镜像地址写入寄存器镜像或相关MSR，保证swaps后汇编可以通过GS快速定位。与此同时，将LSTAR设置为trampoline中的syscall入口地址，使得用户态的syscall指令会直接跳入内核预期的trampoline代码段。SFMask的配置则用于屏蔽sysret/syscall交替过程中不应被继承的标志位，避免返回时出现安全漏洞。

表 4-7 syscall 汇编入口

```
.section .text.trampoline
.code64

.global syscall_entry

syscall_entry:
    swaps
    mov qword ptr gs:[offset __PERCPU_USER_RSP], rsp
    mov rsp, qword ptr gs:[offset __PERCPU_TSS +
{tss_rsp0_offset}]
    sub rsp, 8
    push qword ptr gs:[offset __PERCPU_USER_RSP]
    push r11
    sub rsp, 8
    push rcx
    sub rsp, 2 * 8
    push rax
    .....(此处省略)
    pop rax
    add rsp, 7 * 8
    mov rcx, [rsp - 5 * 8]
    mov r11, [rsp - 3 * 8]
    mov rsp, [rsp - 2 * 8]
    swaps
    sysretq
```

镜像percpu数据在syscall中的用法如上述代码所示。该汇编实现通过swaps切换到内核GS基址，然后将用户rsp暂存到percpu区，取出TSS中的rsp0（指向用户地址空间下的Trap上下文），最后保存用户rsp到栈上，实现用户rsp和TSS.rsp0的交换。syscall路径中不需要保存ss，error code和vector，因此跳过这些部分，而r11和rcx分别被rflags和rip覆盖，需要按rflags和rip的位置保存。接下来的步骤就与trap处理类似了。syscall

返回时，需要将trap上下文中的rflags和rip分别恢复到r11和rcx上，并直接将用户rsp的值恢复到rsp，不需要借助percpu区域。最后通过sysretq返回用户态。

4.5 ACPI 与 PCI 的探测方式

本文在x86_64上以ACPI[8]作为设备发现的入口。先读取RSDP，再解析SDT中的各张表，整理出平台信息。MADT提供中断控制器信息，MCFG提供PCI配置空间信息，SPCR提供串口信息，HPET提供定时器信息，FADT则补充电源和平台基础信息。这一步对x86_64很重要，固件给出的内容往往决定了后续设备能不能被正确识别。本文不假定固定硬件布局，尽量按ACPI表来判断。

在PCI[9]探测上，优先使用MCFG。MCFG给出ECAM区域后，内核按bus范围访问PCI配置空间，逐个找出总线上的设备。如果ACPI表完整，PCI探测就按表中的信息走；如果表缺失或不完整，再退回到平台默认的PCI_ECAM_BASE作为兜底。这样做的目标是保证在标准固件上按标准方式工作，信息不足时也还能启动，不会因为一处表缺失就让整条设备链路中断。AML的作用主要是补足ACPI静态表里没直接写明的信息。某些平台把串口、总线或其他设备信息放在AML方法里，本文在需要时读取这些方法，拿到更完整的设备描述，再把结果纳入统一的设备列表。

实现时APCI/AML和PCI探测中都优先从探测中获取结果，失败后再回退到默认值并在日志中警告回退行为。

本文在域加载阶段按预先定义的列表，逐个加载内核所需的驱动和服务。该过程分为两层：架构无关层处理所有平台共用的加载逻辑，架构相关层定义各平台特有的域。

架构无关层在init_domains()中实现。本文从initrd中读取gzip压缩数据，解压后遍历CPIO档案内容，查找域ELF二进制文件，再依次调用register_domain_elf加载并注册。本文先注册COMMON_INIT_DOMAIN_LIST中定义的通用域，包括缓冲UART、缓冲输入、块设备缓存、设备文件系统、FAT文件系统、管道文件系统、进程文件系统、内存文件系统、系统文件系统、调度器、任务管理、虚拟文件系统和网络栈等。这些域与硬件平台无关，在所有架构上代码和逻辑保持一致。

架构相关层由各架构的ARCH_INIT_DOMAIN_LIST定义。x86_64上的列表包括uart16550串口、本地APIC域、IO APIC域、CMOS RTC、virtio设备等，这些域对应x86_64特有的中断控制器和设备驱动。riscv64上的ARCH_INIT_DOMAIN_LIST则包

括goldfish RTC、PLIC中断控制器、QEMU上的uart16550和virtio设备，以及VisionFive 2平台上的uart8250和SD卡控制器，反映出架构的硬件差异。init_domains()在注册完通用域后，随即遍历ARCH_INIT_DOMAIN_LIST，同样调用register_domain_elf注册这些架构特有的域。这种分层设计确保通用部分在所有架构上只维护一份代码，各架构可在域列表层面灵活定制所需的驱动和服务，避免在通用加载逻辑里散布平台判断。

第 5 章 vDSO 实现与用户态集成

本章介绍vDSO（virtual Dynamically linked Shared Object）在本项目中的实现思路与工程细节，重点讨论构建与产物、内核侧的映射与刷新机制、导出API与用户态解析。

5.1 vdso_crate_template 构建与产物

vdso_crate_template由build_vdso和vdso_helper两个子组件构成。build_vdso负责把vdso_impl编译成可映射的so和api库，vdso_helper负责在vDSO实现里生成vVAR访问宏与数据布局。api库提供了vDSO函数的ABI以省去手写胶水代码，以及供内核调用的加载与映射辅助函数。

首先展示vDSO的构建过程，由项目中的vdso_builder crate完成。vdso_builder只需要提供目标架构、输入与输出路径、创建的so文件名称和api库名称等信息构建BuildConfig，然后调用build_vdso::build_vdso()即可。在build_vdso内部，首先根据输入的配置生成编译命令，将输入项目编译为动态链接库，然后通过解析输入项目的Rust源代码和动态链接库的函数符号生成api库。

下面的代码展示了使用build_vdso构建vDSO的流程。

表 5-1 vdso 构建配置

```
use build_vdso::{build_vdso as build_vdso_tool, BuildConfig};

fn build_vdso_for_arch(target_arch: TargetArch, repo_root:
&Path) {
    let mut config = BuildConfig::new("../vdso/vdso_impl",
"vdso_impl");
    config.arch = target_arch.build_arch().to_string();
    config.out_dir = "../build/vdso".to_string();
    config.so_name = "libvdso".to_string();
    config.toolchain = "nightly-2026-01-23".to_string();
    config.api_lib_name = "vdso_api".to_string();
    build_vdso_tool(&config);
}
```

接下来展示vDSO函数实现，由项目中的vdso_impl定义。根据build_vdso的设计，vdso_impl需要保留一个api模块用于导出公共符号，其余的内部实现可以放到其它模

块中。vdso_impl通过vdso_helper中的vvar_data!与get_vvar_data!声明共享快照区，并在api模块导出__vdso_clock_gettime等符号。部分代码如表5-2所示。

表 5-2 vdso_crate_template 接入与 vVAR 定义

```
vvar_data! {
    seq: usize,
    realtime_ns: usize,
    monotonic_ns: usize,
}

#[unsafe(no_mangle)]
pub extern "C" fn __vdso_clock_gettime(clk: usize, ts: *mut TimeSpec) -> i32
{
    if ts.is_null() {
        return -1;
    }
    match read_clock_timespec(clk) {
        Some(time_spec) => {
            unsafe {
                *ts = time_spec;
            }
            0
        }
        None => -1,
    }
}

pub(crate) fn read_clock_timespec(clock_id: usize) -> Option<TimeSpec> {
    macro_rules! read_snapshot {
        ($field:ident) => {{
            loop {
                let seq = unsafe
            { core::ptr::read_volatile(get_vvar_data!(seq)) };
                if seq & 1 != 0 { core::hint::spin_loop(); continue; }
                let nanos = unsafe
            { core::ptr::read_volatile(get_vvar_data!($field)) };
                let seq_after = unsafe
            { core::ptr::read_volatile(get_vvar_data!(seq)) };
                if seq == seq_after { break Some(TimeSpec { tv_sec: nanos /
            1_000_000_000, tv_nsec: nanos % 1_000_000_000 }); }
            }
        }};
    }
}
```

```

}

match clock_id {
    CLOCK_REALTIME => read_snapshot!(realtime_ns), CLOCK_MONOTONIC =>
        read_snapshot!(monotonic_ns)
    _ => None
}
}

```

本文的实现基于奇偶版本号(seqcount)，内核作为写者，用户程序作为读者。用户态读者使用无锁方式访问vVAR时间快照：先读取一次seq，若为奇数就自旋等待后重试；若为偶数则读取目标时间字段，再读一次seq并比较前后值是否相同。只有两次seq相等，才认为拿到了一组一致的快照，否则重试。本文的实现中使用了volatile读和core::hint::spin_loop()，目的是避免编译器优化或CPU重排序的干扰，强制重新读取值。对于clock_gettime这类高频读取场景，这种模式能显著降低延迟。如果多次重试仍拿不到一致快照，调用方可以退回到系统调用，由内核保证正确结果。

5.2 内核侧 MemIf 实现与 vDSO 装载

vDSO引入需要内核侧MemIf实现来完成物理页分配、页表映射等工作。本文在KernelVdsoMem中实现了vdso_api::MemIf，并在init_vdso中调用vdso_api::map_so完成装载。

本文按照build_vdso的设计，内核首次加载vDSO和后续向用户地址空间映射都使用vdso_api::map_so，区别只在于传入的vspace不同。本文实现时，调用map_so时vspace传入页表token（即写入页表寄存器的值）；MemIf中所有带vspace参数的位置只判断是不是内核token，如果是则就地实现相关功能，如果不是就转而调用task domain接口提供的服务来完成。

表 5-3 内核侧 MemIf 实现与装载入口

```

pub fn init_vdso() {
    VDSO_LOADED.store(true, Ordering::SeqCst);
    let vdso_base = vdso_api::map_so(mem::kernel_page_table_token()) as
    usize;
    unsafe { vdso_api::init_vdso_vtable(vdso_base as u64) };
    VDSO_LOADED.store(false, Ordering::SeqCst);
}

struct KernelVdsoMem;

```

```
#[crate_interface::impl_interface]
impl MemIf for KernelVdsoMem {
    fn valloc(vspace: usize, size: usize) -> *mut u8 { ... }
    fn ppage_alloc(size: usize) -> vdso_api::PhysPagePtr { ... }
    fn map(vspace: usize, vaddr: *mut u8, ppage: vdso_api::PhysPagePtr,
size: usize, flags: vdso_api::MappingFlags) { ... }
    fn change_protect(vspace: usize, vaddr: *mut u8, size: usize, flags:
vdso_api::MappingFlags) { ... }
    fn get_kernel_vaddr(vspace: usize, vaddr: *mut u8) -> *mut u8 { ... }
    fn ppage_clone(ppage: vdso_api::PhysPagePtr) -> vdso_api::PhysPagePtr
{ ... }
}
```

本文在物理页分配时将结果封装为FrameTracker对象，借助Drop trait实现自动回收。为防止FrameTracker被提前释放，ppage_alloc()先将它包装成Arc<FrameTracker>，再用Arc::into_raw()转为裸指针，最后转成usize返回。对应的ppage_clone()则通过Arc::clone()复制一份Arc，返回新的裸指针。

5.3 内核侧刷新策略

内核在首次加载的时候记录了vVAR区地址，随后通过这个地址更新时间快照。内核作为写着，类似地使用seqcount风格的写序列：先把版本号置为奇数表示“写入中”，写入数据字段并执行必要的内存屏障，最后把版本号置为偶数表示“写入完成”。这种写法不使用锁，同时保证用户能读取到正确的值。

表 5-4 内核侧 vVAR 刷新

```
let data = unsafe { &mut *(vvar_base as *mut VvarData) };
let seq = data.seq.wrapping_add(1) | 1;
data.seq = seq;
core::sync::atomic::compiler_fence(core::sync::atomic::Ordering::Release);
data.realtime_ns = realtime_ns;
data.monotonic_ns = monotonic_ns;
core::sync::atomic::compiler_fence(core::sync::atomic::Ordering::Release);
data.seq = seq.wrapping_add(1);
```

vDSO的加载还需要内核在进程初始化时将vDSO的入口地址通过auxv的AT_SYSINFO_EHDR注入到用户态，供用户运行时在启动阶段解析并绑定vDSO提供的符号。若解析失败，用户运行时回退到传统的syscall路径以保证功能完整。

5.4 用户运行时 vDSO 加载

本文把用户态vDSO的加载逻辑集中放在用户运行时(userlib)里。userlib通过解析程序初始栈上的auxv获取vDSO ELF的基址，然后初始化vdso_api的vtable。之后调用方就能直接使用vDSO导出的快速路径接口（例如__vdso_clock_gettime）。当vDSO不可用或调用失败时，userlib会退回到传统的syscall实现。

表5-5展示userlib如何解析auxv、初始化vDSO基址并优先使用vDSO快路径。

表 5-5 userlib vDSO 加载与调用

```
static VDSO_BASE: AtomicUsize = AtomicUsize::new(0);
static VDSO_READY: AtomicUsize = AtomicUsize::new(0);

pub fn init_from_stack(argc_ptr: usize) {
    if let Some(base) = parse_vdso_base(argc_ptr) {
        init(base);
    }
}

pub fn init(base: usize) {
    if !is_valid_vdso_base(base) { return; }
    VDSO_BASE.store(base, Ordering::Release);
    unsafe { vdso_api::init_vdso_vtable(base as u64) };
    VDSO_READY.store(1, Ordering::Release);
}

pub fn clock_gettime_vdso(clk: usize, ts: &mut TimeSpec) -> bool {
    if VDSO_READY.load(Ordering::Acquire) != 0 &&
    VDSO_BASE.load(Ordering::Acquire) != 0 {
        let mut vdso_ts = vdso_api::TimeSpec::default();
        let ret = vdso_api::__vdso_clock_gettime(clk, &mut vdso_ts as *mut
_);
        if ret == 0 {
            ts.tv_sec = vdso_ts.tv_sec;
            ts.tv_nsec = vdso_ts.tv_nsec;
            return true;
        }
    }

    false
}

pub fn clock_gettime(clk: usize, ts: &mut TimeSpec) -> bool {
    if clock_gettime_vdso(clk, ts) { return true; }
```

```
sys_clock_gettime(clk, ts as *mut TimeSpec as *mut u8) == 0  
}
```

在进程启动时userlib从用户栈上解析AT_SYSINFO_EHDR并缓存vDSO基址，然后用vdso_api::init_vdso_vtable()把vDSO导出的符号绑定到用户态可调用的函数指针或跳转表。当快路径调用失败时，回退到sys_clock_gettime系统调用，保证功能完整。

第6章 验证与评估

本章基于一次完整运行的实验日志，系统性地说明将系统迁移到x86_64平台后在功能性和性能方面的验证结果。本章依次讨论测试设计、功能验证、性能度量，结论部分总结主要发现和后续工作方向。

6.1 测试设计

在测量用户态读取时钟的快路径vDSO相对于传统系统调用路径raw syscall之前，先把两者获取的时间的差距检查当作功能测试。测试代码会先做16次检查，每次先调用vDSO，再调用raw syscall，然后把结果都转成纳秒并算出绝对差值。如果差值大于1ms，就直接判定失败。这样做的目的是：先确认vDSO和raw syscall都能返回正确时间，再进入后面的性能测试。

表 6-1 vDSO 功能测试代码

```
fn verify_correctness(threshold_us: u64) -> Result<(), &'static str> {
    for i in 0..VERIFY_ITERS {
        let mut vdso_ts = TimeSpec::default();
        let mut raw_ts = TimeSpec::default();
        assert_eq!(clock_gettime_vdso(CLOCK_MONOTONIC, &mut vdso_ts), 0);
        assert_eq!(clock_gettime_raw(CLOCK_MONOTONIC, &mut raw_ts), 0);

        let vdso_ns = timespec_to_ns(&vdso_ts);
        let raw_ns = timespec_to_ns(&raw_ts);
        let diff_ns = if vdso_ns > raw_ns { vdso_ns - raw_ns } else { raw_ns
- vdso_ns };
        let diff_us = (diff_ns + 999) / 1000;
        if diff_us > threshold_us as u128 {
            println!("[VDSO_VERIFY] iter={} vdso_ns={} raw_ns={}
diff_us={}", i, vdso_ns, raw_ns, diff_us);
            return Err("vdso vs raw difference exceeds threshold");
        }
    }
    Ok(())
}
```

本文做性能测试时，用一个独立的时间源来计时，确保测量过程本身不会干扰被测试调用。方法是不直接对单次取时间调用本身计时，而是设计了一个批量测量窗口：在窗口开启前先调一次系统调用记下起始纳秒时间，然后在窗口里连续跑很多次被

测的取时间调用，结束后再调一次系统调用记下结束纳秒时间。这样用窗口总耗时除以调用次数，就得到单次平均耗时，同时把系统调用等额外开销平均分摊下去，减小计时偏差。

为了让缓存不干扰结果，每组正式测量前都先跑64次预热，让指令和数据尽可能进缓存，也让分支预测等硬件状态稳定下来。同时，为了避免编译器把测量循环优化掉（比如觉得循环里没什么用就直接删掉），测量时用`core::hint::black_box`把被测调用包起来，强制编译器保留调用，保证真的执行了那么多遍。

调用次数选了64、128、256、512、1024这几档，从少到多都覆盖到，方便看窗口变大后单次平均耗时会不会更稳定，以及不同调用次数下有没有缓存效应或别的波动。每组实验独立重复5次，然后算平均值和标准差，这样数据相对稳定。

6-2 窗口测量代码

```
fn measure_groups(groups: &[usize]) -> alloc::vec::Vec<PerfResult> {
    let mut results: alloc::vec::Vec<PerfResult> = alloc::vec::Vec::new();
    for &g in groups.iter() {
        for _ in 0..WARMUP_ITERS {
            let _ = { let mut ts = TimeSpec::default();
            assert_eq!(clock_gettime_vdso(CLOCK_MONOTONIC, &mut ts), 0); ts };
            let _ = { let mut ts = TimeSpec::default();
            assert_eq!(clock_gettime_raw(CLOCK_MONOTONIC, &mut ts), 0); ts };
        }

        let mut vdso_avgs: alloc::vec::Vec<u128> = alloc::vec::Vec::new();
        let mut raw_avgs: alloc::vec::Vec<u128> = alloc::vec::Vec::new();

        for rep in 0..REPEATS {
            let vdso_elapsed = measure_vdso_ns(g);
            let raw_elapsed = measure_raw_ns(g);
            let vdso_avg = vdso_elapsed / g as u128;
            let raw_avg = raw_elapsed / g as u128;
            let speedup = if vdso_avg == 0 { 0.0 } else { raw_avg as f64 /
vdso_avg as f64 };

            println!("[VDSO_BENCH] repeat={} samples={} unit={}
vdso_elapsed={} vdso_avg={} raw_elapsed={} raw_avg={} speedup={:.2}x",
                    rep + 1, g, BENCH_UNIT, vdso_elapsed, vdso_avg, raw_elapsed,
                    raw_avg, speedup);

            vdso_avgs.push(vdso_avg);
```

```

        raw_avgs.push(raw_avg);
    }

    let (vdso_mean, vdso_std) = compute_stats(&vdso_avgs);
    let (raw_mean, raw_std) = compute_stats(&raw_avgs);
    let agg_speedup = if vdso_mean == 0 { 0.0 } else { raw_mean as f64 /
vdso_mean as f64 };

    println!("[VDSO_AGG] samples={} repeats={} vdso_mean={} vdso_std={}
raw_mean={} raw_std={} speedup_mean={:.2}x",
            g, REPEATS, vdso_mean, vdso_std, raw_mean, raw_std,
            agg_speedup);

    results.push(PerfResult {
        samples: g,
        unit: BENCH_UNIT,
        vdso_elapsed: vdso_mean * g as u128,
        vdso_avg: vdso_mean,
        raw_elapsed: raw_mean * g as u128,
        raw_avg: raw_mean,
        speedup: agg_speedup,
    });
}
results
}

```

整体上，这套测试分成两步。第一步先做功能检查，重点看vDSO和raw syscall的结果是不是在可接受范围内。第二步再做性能比较，重点看两条路径的单次平均耗时差多少。也就是说，vDSO与raw syscall时间差距首先被当作功能测试项处理，只有在功能确认通过后，才继续看性能数据。

表 6-3 计时基准代码

```

fn monotonic_now_ns() -> u128 {
    let mut ts = TimeSpec::default();
    assert_eq!(clock_gettime_raw(CLOCK_MONOTONIC, &mut ts), 0);
    timespec_to_ns(&ts)
}

```

这段基准代码很简单，只做一件事：调用raw syscall读取单调时钟，再把结果换算成纳秒。它的作用是给窗口前后提供同一个时间基准，避免用被测调用自己的返回值来回算耗时。这样得到的结果更稳定，也更适合比较vDSO和raw syscall在同一窗口里的平均表现。

本文把测量误差分成了两类。一类是统计误差，来自单次运行的样本数量不够；另一类是系统性误差，来自调度、后台干扰或者测量代码本身的开销。为了降低统计误差，本文对每个样本组都做多次完整运行，再算均值和方差。为了减小系统性误差，测试在单核上运行，除了测试之外后台没有其它进程，并且保持QEMU的配置参数不变。

6.2 功能验证与运行证据

功能验证基于系统从引导到进入用户态测试并成功呈现交互shell的日志序列。表6-4摘录代表性日志片段，用以证明引导、内存与地址空间初始化、域注册与加载、以及用户态测试的顺利完成。

表 6-4 运行日志节选

[0] [x86_boot] main_entry magic=0x2badb002 mbi=0x9500
[0] Frame allocator init success
[0] Build kernel address space success
[0] ++++ setup interrupt +++
[0] ++++ setup interrupt done, enable:true +++
[0] <register domain>: scheduler, logger, fatfs, ramfs, devfs, procfs, sysfs, pipefs, domainfs, vfs, task, syscall, uart16550, local_apic, io_apic, cmos_rtc, virtio_net
[0] Load domains done
...
[syscall_all] start
[sys_poll] PASS
[sys_fs] PASS
[sys_link] PASS
[sys_proc] PASS
[sys_random] PASS
[sys_time] PASS
[sys_vdso_time_compare] PASS
[syscall_all] PASS
[Init] all tests passed
Alien:/#

本文认为，这些日志片段给出了整条链路的证据。启动阶段验证了引导、内核早期初始化、内存初始化、中断初始化正确完成。域注册与加载片段验证了系统模块化加载逻辑和依赖解析。用户态测试通过则验证了系统调用转发、用户态库和运行时环

境工作的正确性。虽然日志不可能穷尽所有边界情况，但它的连贯性和顺序性已足够作为功能完整性的主要证据。

6.3 性能度量与分析

功能验证通过后，本文开始测量性能。对vDSO和raw syscall两种方式，分别在不同样本组（64、128、256、512、1024）上用窗口法测量平均单次耗时。下面列出运行日志中记录的原始测量数据，作为后续统计分析的基础。

表 6-5 vdsO 与 syscall 性能对比表

样本数	重复次数	vDSO 平均(ns)	vDSO 标准差(ns)	raw 平均(ns)
64	5	1327	512	38860
128	5	511	64	35357
256	5	363	64	37914
512	5	282	16	36551
1024	5	286	32	45534

表6-5中的数据显示，vDSO平均单次耗时在几百到上千纳秒之间，而raw syscall的平均耗时在几万纳秒，vDSO比raw快出两个数量级。每组内部重复测量5次并计算均值与标准差后，本文发现：较大样本组（256、512、1024）中raw的均值更稳定，但仍有几千纳秒的波动。vDSO的标准差相对较小却不恒定，说明虽然vDSO单次开销很小，其测量值仍然容易受系统抖动和基准打点误差影响。基于以上分析，本文后续将尝试使用更大的样本数稀释误差，并尝试更快的时间获取方式（不再限制单位，考虑直接读TSC计数值），最后在真实硬件上复验。

结 论

本文以工程实现为主，围绕将Alien系统从RISC-V平台迁移到x86_64并引入vDSO的设计展开设计和实现工作。在不破坏既有RISC-V路径的前提下，我们提出并实现了针对x86_64的平台抽象，完成了启动与早期初始化、内存管理、中断与syscall适配等关键子系统的移植工作，构建了域加载与设备探测的链路，并实现了vDSO/vVAR的引入以提供用户态的低延迟快速路径和备用回退机制。初步验证表明，本文提出的架构抽象有助于减少多架构带来的维护负担，vDSO在常见的时钟查询场景中带来了可观的延迟改善，而域隔离与驱动加载设计在多核环境下保持了基本的可用性与可恢复性。尽管当前实现仍以最小可运行目标为导向，存在设备支持不全、某些子系统需进一步扩展及更严格形式化验证不足等局限，但本研究为在多架构环境下实现轻量级隔离内核提供了切实的工程路径与实践经验，并为后续在协作调度、用户态中断、vDSO深度优化与更广泛设备支持等方向的研究奠定了基础。

后续工作将围绕三个相互关联的方向展开：

1. 将vsched2的协作调度机制融入域生命周期，并建立可回退的混合调度策略
2. 引入用户态中断以在保持隔离性的前提下缩短I/O与事件处理延迟
3. 在x86_64物理硬件上完成部署与长期验证。

这些扩展将与vDSO的深度优化协同推进，逐步构筑一个在真实多核硬件上兼具严格隔离、低延迟通信和工程可维护性的多架构轻量级隔离内核体系。

参考文献

- [1] Godones. Alien: A simple operating system implemented in Rust[CP/OL]. [2026-05-19]. <https://github.com/Godones/Alien>.
- [2] Rust OSDev Community. acpi: A library for parsing ACPI tables and AML[CP/OL]. [2026-05-18]. <https://github.com/rust-osdev/acpi>.
- [3] x2apic Developers. x2apic: A Rust interface to the x2apic interrupt architecture (version 0.4.1)[CP/OL]. [2026-05-18]. <https://docs.rs/x2apic/0.4.1/>.
- [4] x86_64 Contributors. x86_64: Support for x86_64 specific instructions, registers, and structures[CP/OL]. [2026-05-18]. https://docs.rs/x86_64/.
- [5] ArceOS Community. ArceOS: A modular operating system framework in Rust[CP/OL]. [2026-05-18]. <https://github.com/arceos/arceos>.
- [6] rosy233333. vdso_crate_template[CP/OL]. [2026-05-18]. https://github.com/rosy233333/vdso_crate_template.
- [7] Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer ‘s Manual Combined Volumes 3A, 3B, 3C, and 3D: System Programming Guide[R/OL]. (2026-03-31)[2026-05-19]. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [8] UEFI Forum. Advanced Configuration and Power Interface (ACPI) Specification, Version 6.5[S/OL]. (2022-08-29)[2026-05-19]. <https://uefi.org/specifications>.
- [9] PCI-SIG. PCI Firmware Specification Revision 3.2[S/OL]. (2015-01-26)[2026-05-19]. <https://pcisig.com/specifications>.
- [10] GNU GRUB Project. Multiboot Specification (version 0.6.96)[S/OL]. [2026-05-19]. <https://www.gnu.org/software/grub/manual/multiboot/multiboot.html>.
- [11] RISC-V International. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 20211203[S/OL]. (2021-12-03)[2026-05-19]. <https://riscv.org/technical/specifications/>.
- [12] Linux Kernel Community. vdso(7) — Linux manual page[EB/OL]. (2026-03-20)[2026-05-19]. <https://manpages.debian.org/testing/manpages/vdso.7.en.html>.
- [13] Klein G, Elphinstone K, Heiser G, et al. seL4: formal verification of an OS kernel[C]//Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP 2009). Big Sky, Montana, USA: ACM, 2009: 207-220.
- [14] Liedtke J. On μ -kernel construction[C]//Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP 1995). Copper Mountain Resort, CO: ACM, 1995: 237-250.
- [15] Jung R, Jourdan J H, Krebbers R, et al. RustBelt: securing the foundations of the Rust programming language[J]. Proceedings of the ACM on Programming Languages (POPL), 2018, 2: 1-34.
- [16] Klabnik S, Nichols C. The Rust Programming Language[M]. 2nd ed. San Francisco: No Starch Press, 2023.
- [17] The Rust Project. The Rust Reference[EB/OL]. [2026-05-19]. <https://doc.rust-lang.org/reference/>.
- [18] Kleen A. x86_64: Add vDSO for x86-64 with gettimeofday/clock_gettime/getcpu[EB/OL]. (2007-05-01)[2026-05-21]. <https://lkml.indiana.edu/hypermail/linux/kernel/0705.0/0071.html>.
- [19] Frascino V. Unify vDSOs across more architectures[EB/OL]. (2019-06-21)[2026-05-21]. <https://lwn.net/Articles/791685/>.

- [20] Schnelle S. s390: convert to GENERIC_VDSO[EB/OL]. (2020-08-03)[2026-05-21]. <https://lkml.org/lkml/2020/8/3/622>.
- [21] Larabel M. MSEAL Protection Of System Mappings Merged For Linux 6.15[EB/OL]. (2025-04-04)[2026-05-21]. <https://www.phoronix.com/news/Linux-6.15-MSEAL-System-Mapping>.
- [22] scumjr. dirtycow-vdso: PoC for Dirty COW (CVE-2016-5195) vDSO container escape[EB/OL]. (2016)[2026-05-21]. <https://github.com/scumjr/dirtycow-vdso>.